

Google App Engine

Google App Engine (GAE) is a platform-as-a-service product that provides web app developers and enterprises with access to Google's scalable hosting and tier 1 internet service.

GAE requires that applications be written in Java or Python, store data in Google Big table and use the Google query language. Noncompliant applications require modification to use GAE.

GAE provides more infrastructure than other scalable hosting services, such as Amazon Elastic Compute Cloud (EC2). GAE also eliminates some system administration and development tasks to make writing scalable applications easier.

Google provides GAE free up to a certain amount of use for the following resources:

- ↯ processor (CPU)
- ↯ storage
- ↯ application programming interface (API) calls
- ↯ concurrent requests

Users exceeding the per-day or per-minute rates can pay for more of these resources.

Key features of GAE include the following:

API selection. GAE has several built-in APIs, including the following five:

1. Blobstore for serving large data objects;
2. GAE Cloud Storage for storing data objects;
3. Page Speed Service for automatically speeding up webpage load times;
4. URL Fetch Service to issue HTTP requests and receive responses for efficiency and scaling; and
5. Memcache for a fully managed in-memory data store.

Development of scalable web application on Google cloud

How to start your lab and sign in to the Google Cloud Console

1. Click the **Start Lab** button. If you need to pay for the lab, a pop-up opens for you to select your payment method. On the left is the **Lab Details** panel with the following:

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

- ↗ The **Open Google Console** button
 - ↗ Time remaining
 - ↗ The temporary credentials that you must use for this lab
 - ↗ Other information, if needed, to step through this lab
2. Click **Open Google Console**. The lab spins up resources, and then opens another tab that shows the **Sign in** page.

Tip: Arrange the tabs in separate windows, side-by-side.

Note: If you see the **Choose an account** dialog, click **Use Another Account**.

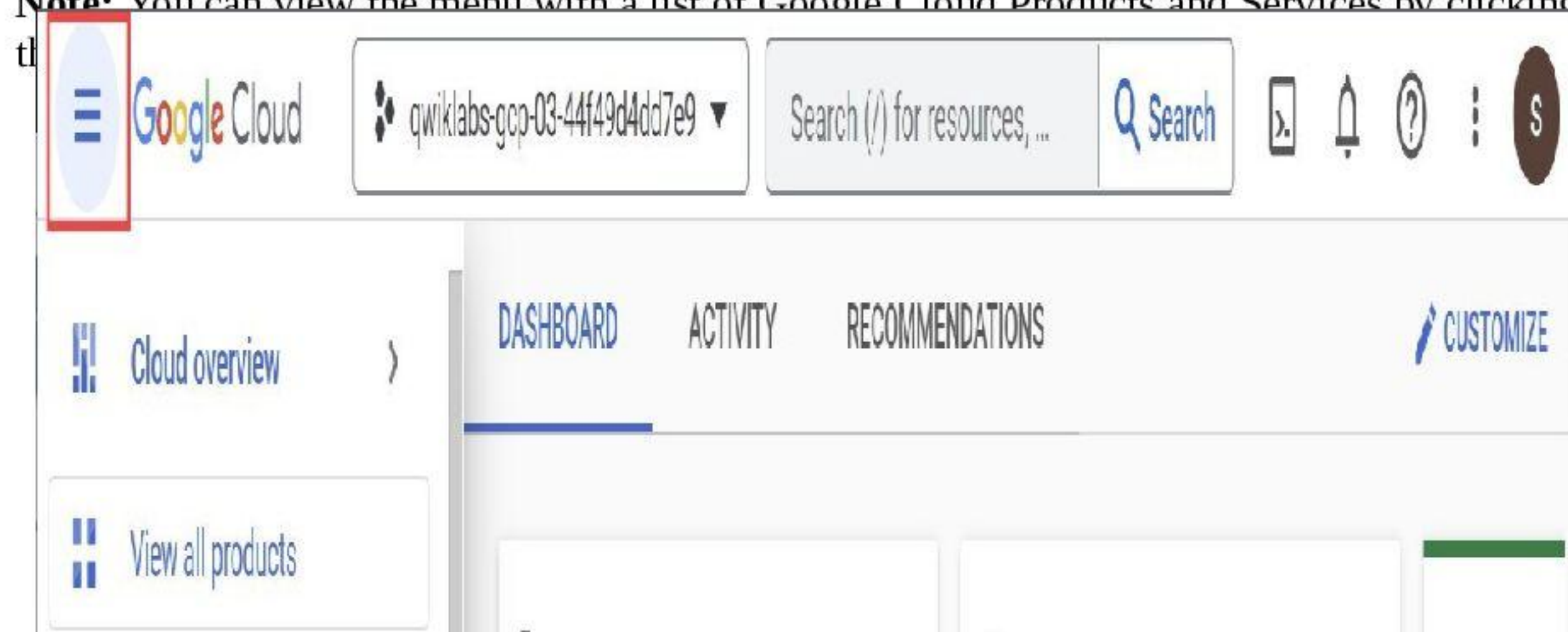
3. If necessary, copy the **Username** from the **Lab Details** panel and paste it into the **Sign in** dialog. Click **Next**.
4. Copy the **Password** from the **Lab Details** panel and paste it into the **Welcome** dialog. Click **Next**.

Important: You must use the credentials from the left panel. Do not use your Google Cloud Skills Boost credentials. **Note:** Using your own Google Cloud account for this lab may incur extra charges.

5. Click through the subsequent pages:
 - ↗ Accept the terms and conditions.
 - ↗ Do not add recovery options or two-factor authentication (because this is a temporary account).
 - ↗ Do not sign up for free trials.

After a few moments, the Cloud Console opens in this tab.

Note: You can view the menu with a list of Google Cloud Products and Services by clicking the



Activate Cloud Shell

Cloud Shell is a virtual machine that is loaded with development tools. It offers a persistent 5GB home directory and runs on the Google Cloud. Cloud Shell provides command-line access to your Google Cloud resources.

1. Click **Activate Cloud Shell**  at the top of the Google Cloud console.

When you are connected, you are already authenticated, and the project is set to

Your Cloud Platform project in this session is set to YOUR_PROJECT_ID for this session. Cloud Shell is a command-line tool for Google Cloud. It comes pre-installed on Cloud Shell and supports tab-completion.

2. (Optional) You can list the active account name with this command:

```
gcloud auth list
```

Copied!

content_copy

3. Click **Authorize**.

4. Your output should

```
ACTIVE: *
ACCOUNT: student-01-xxxxxxxxxxxx@gwiklabs.net
To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

Output:

5. (Optional) You can list the project ID with this command:

```
gcloud config list project
```

Copied!

content_copy

Output:

```
[core]
project = <project_ID>
```

Example output:

```
[core]
project = qwiklabs-gcp-44776a13dea667a6
```

Note: For full documentation of gcloud, in Google Cloud, refer to [the gcloud CLI overview guide](#).

Set your region and zone

Certain Compute Engine resources live in regions and zones. A region is a specific geographical location where you can run your resources. Each region has one or more zones.

Learn more about regions and zones and see a complete list in [Regions & Zones documentation](#).

Run the following gcloud commands in Cloud Console to set the default region and zone for your lab:

```
gcloud config set compute/zone "place_holder_text"
export ZONE=$(gcloud config get compute/zone)
gcloud config set compute/region
"place_holder_text"
export REGION=$(gcloud config get compute/region)
Copied!
content_copy
```

Task 1. Enable Compute Engine API

Next, enable the [Compute Engine API](#).

Execute the following to enable the Compute Engine API:

```
gcloud services enable compute.googleapis.com
Copied!
content_copy
```

Task 2. Create Cloud Storage bucket

You will use a Cloud Storage bucket to house your built code as well as your startup scripts.

- From within Cloud Shell, execute the following to create a new Cloud Storage bucket:

```
gsutil mb gs://fancy-store-$DEVSHELL_PROJECT_ID
```

Copied!

content_copy

Note: Use of the \$DEVSHELL_PROJECT_ID environment variable within Cloud Shell is to help ensure the names of objects are unique. Since all Project IDs within Google Cloud must be unique, appending the Project ID should make other names unique as well.

Click **Check my progress** to verify the objective.

Create Cloud Storage bucket

Check my progress

Task 3. Clone source repository

Use the existing Fancy Store ecommerce website based on the `monolith-to-microservices` repository as the basis for your website.

Clone the source code so you can focus on the aspects of deploying to Compute Engine. Later on in this lab, you will perform a small update to the code to demonstrate the simplicity of updating on Compute Engine.

1. Clone the source code and then navigate to the `monolith-to-microservices` directory:

```
git clone https://github.com/googlecode labs/monolith-to-microservices.git
```

Copied!

content_copy

cd

~/monolith

-to-

microservi

ces

Copied!

content_copy

2. Run the

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
./setup.sh  
Copied!  
content_copy  
It will take  
a few  
minutes  
for this  
script to  
finish.
```

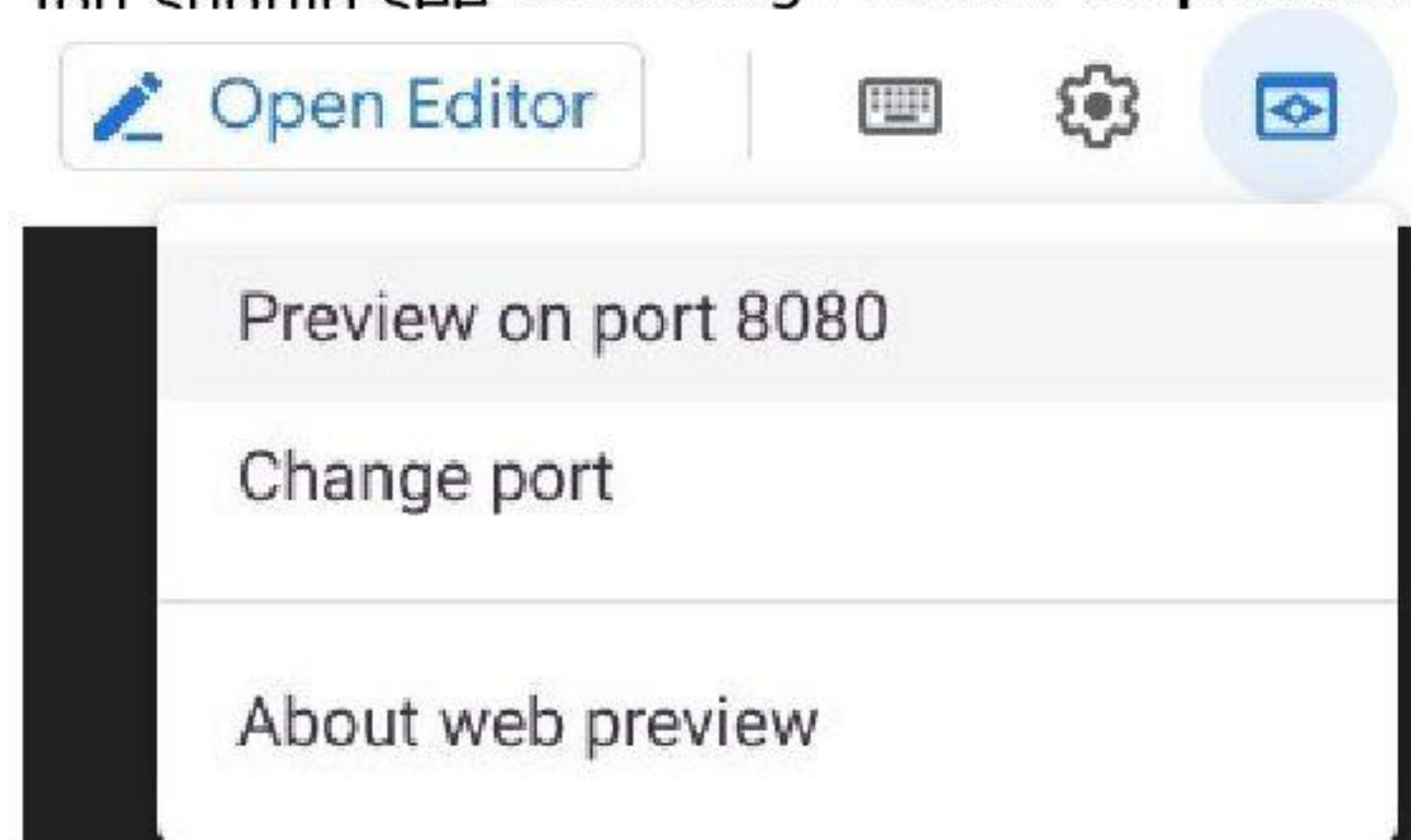
3. Once completed, ensure Cloud Shell is running a compatible nodeJS version with the following command:

```
nvm install --lts  
Copied!  
content_copy
```

4. Next, run the following to test the application, switch to the `microservices` directory, and start the web server:

```
Products microservice listening on port 8082!  
Frontend microservice listening on port 8080!  
Orders microservice listening on port 8081!
```

5. Preview your application by clicking the **web preview** icon, then selecting **Preview on port 8080**.



This opens a new window where you can see the frontend of Fancy Store.

Note: Within the Preview option, you should be able to see the Frontend; however, the Products and Orders functions will not work, as those services are not yet exposed.

6. Close this window after viewing the website and then press CTRL+C in the terminal window to stop the web server process.

Task 4. Create Compute Engine instances

Now it's time to start deploying some Compute Engine instances!

In the following steps you will:

1. Create a startup script to configure instances.
2. Clone source code and upload to Cloud Storage.
3. Deploy a Compute Engine instance to host the backend microservices.
4. Reconfigure the frontend code to utilize the backend microservices instance.
5. Deploy a Compute Engine instance to host the frontend microservice.
6. Configure the network to allow communication.

Create the startup script

A startup script will be used to instruct the instance what to do each time it is started. This way the instances are automatically configured.

1. In Cloud Shell, run the following command to create a file called startup-script.sh:

```
touch ~/monolith-to-microservices/startup-script.sh
```

Copied!

content_copy

2. Click

 Open Editor



3. Shell

folder

on
to
ope
n

the

Cod

e

Edit

4. Add the following code to the startup-script.sh file. You will edit some of the code after it's added:

```
#!/bin/bash
# Install logging monitor. The monitor will automatically pick up logs
sent to
# syslog.
curl -s "https://storage.googleapis.com/signals-agents/logging/google-
fluentd-install.sh" | bash
service google-fluentd restart &
# Install dependencies from apt
apt-get update
apt-get install -yq ca-certificates git build-essential supervisor
psmisc
# Install nodejs
mkdir /opt/nodejs
curl https://nodejs.org/dist/v16.14.0/node-v16.14.0-linux-x64.tar.gz |
tar xvzf - -C /opt/nodejs --strip-components=1
ln -s /opt/nodejs/bin/node /usr/bin/node
ln -s /opt/nodejs/bin/npm /usr/bin/npm
# Get the application source code from the Google Cloud Storage bucket.
mkdir /fancy-store
gsutil -m cp -r gs://fancy-store-[DEVSHIELD_PROJECT_ID]/monolith-to-
microservices/microservices/* /fancy-store/
# Install app dependencies.
cd /fancy-store/
npm install
# Create a nodeapp user. The application will run as this user.
useradd -m -d /home/nodeapp nodeapp
chown -R nodeapp:nodeapp /opt/app
# Configure supervisor to run the node app.
cat >/etc/supervisor/conf.d/node-app.conf << EOF
[program:nodeapp]
directory=/fancy-store
command=npm start
autostart=true
autorestart=true
user=nodeapp
environment=HOME="/home/nodeapp",USER="nodeapp",NODE_ENV="production"
stdout_logfile=syslog
stderr_logfile=syslog
EOF
supervisorctl reread
supervisorctl update
Copied!
```

content_copy

5. Find the text

Project ID

The line of code within startup-script.sh should now resemble the file and

```
gs://fancy-store-Project ID/monolith-to-microservices/microservices/*
/fancy-store/
```

6. **Save** the startup-script.sh file, but do not close it yet.

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

7. Look at the bottom right of Cloud Shell Code Editor, and ensure "End of Line Sequence" is set to "LF" and not "CRLF".



- ↗ If this is set to CRLF, click **CRLF** and then select **LF** in the drop down.
- ↗ If this is already set to **LF**, then leave as is.

8. Close the startup-script.sh file.

9. Return to Cloud Shell Terminal and run the following to copy the startup-script.sh file into your bucket:

```
gsutil cp ~/monolith-to-microservices/startup-script.sh gs://fancy-store-$DEVSHIELD_PROJECT_ID
```

Copied!

content_copy

It will now

be

accessible

at:

[https://storage.googleapis.com/\[BUCKET_NAME\]/startup-script.sh](https://storage.googleapis.com/[BUCKET_NAME]/startup-script.sh)

[BUCKET_NAME] represents the name of the Cloud Storage bucket. This will only be viewable by authorized users and service accounts by default, so inaccessible through a web browser. Compute Engine instances will automatically be able to access this through their service account.

The startup script performs the following tasks:

- ↗ Installs the Logging agent. The agent automatically collects logs from syslog.
- ↗ Installs Node.js and Supervisor. Supervisor runs the app as a daemon.
- ↗ Clones the app's source code from Cloud Storage Bucket and installs dependencies.

- ↗ Configures Supervisor to run the app. Supervisor makes sure the app is restarted if it exits unexpectedly or is stopped by an administrator process. It also sends the app's stdout and stderr to syslog for the Logging agent to collect.

Copy code into the Cloud Storage bucket

When instances launch, they pull code from the Cloud Storage bucket, so you can store some configuration variables within the `.env` file of the code.

Note: You could also code this to pull environment variables from elsewhere, but for demonstration purposes this is a simple method to handle configuration. In production, environment variables would likely be stored outside of the code.

1. Copy the cloned code into your bucket:

```
cd ~  
rm -rf monolith-to-microservices/*/node_modules  
gsutil -m cp -r monolith-to-microservices gs://fancy-store-  
$DEVSHELL_PROJECT_ID/
```

Copied!

content_copy

Note: The `node_modules` dependencies directories are deleted to ensure the copy is as fast and efficient as possible. These are recreated on the instances when they start up.

Click **Check my progress** to verify the objective.

Copy startup script and code to Cloud Storage bucket

Check my progress

Deploy the backend instance

The first instance to be deployed will be the backend instance which will house the Orders and Products microservices.

Note: In a production environment, you may want to separate each microservice into their own instance and instance group to allow them to scale independently. For demonstration purposes, both backend microservices (Orders & Products) will reside on the same instance and instance group.

- ↗ Execute the following command to create an `e2-standard-2` instance that is configured to use the startup script. It is tagged as a backend instance so you can apply specific firewall rules to it later:

```
gcloud compute instances create backend \  
  --zone=$ZONE \  
  --machine-type=e2-standard-2 \  
  --tags=backend \  
  --metadata=startup-script-url=https://storage.googleapis.com/fancy-  
store-$DEVSHELL_PROJECT_ID/startup-script.sh
```

Copied!

content_copy

Configure a connection to the backend

Before you deploy the frontend of the application, you need to update the configuration to point to the backend you just deployed.

1. Retrieve the external IP address of the backend with the following command, look under the EXTERNAL_IP tab for the backend instance:

```
gcloud compute instances list
```

Copied!

content_copy

```
NAME: backend
ZONE: zone
MACHINE_TYPE: e2-standard-2
PREEMPTIBLE:
INTERNAL_IP: 10.142.0.2
EXTERNAL_IP: 35.237.245.193
STATUS: RUNNING
```

2. **Copy the External IP** for the backend.
3. In the Cloud Shell Explorer, navigate to `monolith-to- microservices > react-app`.
4. In the Code Editor, select **View > Toggle Hidden Files** in order to see the `.env` file.

In the next step, you edit the `.env` file to point to the External IP of the backend. **[BACKEND_ADDRESS]** represents the External IP address of the backend instance determined from the above `gcloud` command.

```
REACT_APP_ORDERS_URL=http://[BACKEND_ADDRESS]:8081/api/orders
REACT_APP_PRODUCTS_URL=http://[BACKEND_ADDRESS]:8082/api/products
```

6. **Save** the file.
7. In Cloud Shell, run the following to rebuild `react-app`, which will update the frontend code:

```
cd ~/monolith-to-microservices/react-app
```


CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
npm install && npm run-script build
```

Copied!

```
content_copy
```

8.

Then

copy

the

applica

tion

code

into the

Cloud

Storage

bucket:

Deploy the frontend

Instance

Now that the code is configured, deploy the frontend instance.

to-

microservi

Execute the following to deploy the frontend instance with a similar

command as before. This instance is tagged as frontend for firewall

purposes:

```
cp -r
```

```
monolith-
```

```
gcloud compute instances create frontend \
```

```
zones=$ZONE \
```

```
machine-type=e2-standard-2 \
```

```
tags=frontend \
```

```
startup-script-url=https://storage.googleapis.com/fancy-
```

```
$DEVSHELL_PROJECT_ID/startup-script.sh
```

```
PROJECT_ID
```

```
content_copy
```

Noted!

deployment

command

and startup

script is used

with both the

frontend and

backend

instances for simplicity, and because the code is configured to launch all microservices by

default. As a result, all microservices run on both the frontend and backend in this sample. In a

production environment you'd only run the microservices you need on each component.

Create firewall rules to allow access to port 8080 for the frontend,

and ports 8081-8082 for the backend. These firewall commands

use the tags assigned during instance creation for application:

```
gcloud compute firewall-rules create fw-fe \
```

```
--allow tcp:8080 \
```

```
--target-tags=frontend
```

Copied!

Configure the

network

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul®
University

NAAC A++

```
content_copy
gcloud compute firewall-rules create fw-be \
  --allow tcp:8081-8082 \
  --target-tags=backend
```

Copied!

content_copy

The
website
should
now be
fully
functional.

2. In order to navigate to the external IP of the frontend, you need to know the address. Run the following and look for the EXTERNAL_IP of the frontend instance:

```
NAME: backend
ZONE: us-central1-f
MACHINE_TYPE: e2-standard-2
PREEMPTIBLE:
INTERNAL_IP: 10.128.0.2
EXTERNAL_IP: 34.27.178.79
STATUS: RUNNING
NAME: frontend
ZONE: us-central1-f
MACHINE_TYPE: e2-standard-2
PREEMPTIBLE:
INTERNAL_IP: 10.128.0.3
EXTERNAL_IP: 34.172.241.242
STATUS: RUNNING
```

It may take a couple minutes for the instance to start and be configured.

3. Wait 3 minutes and then open a new browser tab and browse to `http://[FRONTEND_ADDRESS]:8080` to access the website, where [FRONTEND_ADDRESS] is the frontend EXTERNAL_IP determined above.
4. Try navigating to the **Products** and **Orders** pages; these should now work.

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

Fancy Store

[Home](#)
[Products](#)
[Orders](#)



MS - Vintage Typewriter - \$67.99



MS - Vintage Camera Lens - \$12.49



MS - Home Barista Kit - \$124



MS - Terrarium - \$36.45



MS - Film Camera - \$2245



MS - Vintage Record Player - \$65.5



MS - Metal Camping Mug - \$24.33



MS - City Bike - \$789.5



MS - Air Plant - \$12.3

Click **Check my progress** to verify the objective.

Deploy instances and configure network

[Check my progress](#)

Task 5. Create managed instance groups

To allow the application to scale, managed instance groups will be created and will use the frontend and backend instances as Instance Templates.

A managed instance group (MIG) contains identical instances that you can manage as a single entity in a single zone. Managed instance groups maintain high availability of your apps by proactively keeping your instances available, that is, in the RUNNING state. You will be using managed instance groups for your frontend and backend instances to provide autohealing, load balancing, autoscaling, and rolling updates.

Create instance template from source instance

Before you can create a managed instance group, you have to first create an instance template that will be the foundation for the group. Instance templates allow you to define the machine type, boot disk image or container image, network, and other instance properties to use when creating new VM instances. You can use instance templates to create instances in a managed instance group or even to create individual instances.

To create the instance template, use the existing instances you created previously.

1. First, stop both instances:

```
gcloud compute instances stop frontend --zone=$ZONE
```

Copied!

```
content_copy  
gcloud  
compute  
instances  
stop  
backend --  
zone=$ZONE
```

Copied!

```
content_copy
```

2. The
n,
crea
te
the
insta
nce
tem
plat
e
from
each

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
gcloud compute instance-templates list
```

Copied!

content_copy

Example

output:

```
NAME: fancy-be
MACHINE_TYPE: e2-standard-2
PREEMPTIBLE:
CREATION_TIMESTAMP: 2023-07-25T14:52:21.933-07:00
NAME: fancy-fe
MACHINE_TYPE: e2-standard-2
PREEMPTIBLE:
CREATION_TIMESTAMP: 2023-07-25T14:52:15.442-07:00
```

4. With the instance templates created, delete the backend vm to save resource space.

```
gcloud compute instances delete backend --zone=$ZONE
```

Copied!

content_copy

5. Type

and

ente

ry

whe

n

pro

mp

Create managed instance

group.

Normally, you could delete the frontend vm as well, but you will use it to update the instance template later in the lab.

1. Next, create two managed instance groups, one for the frontend and one for the backend.

```
gcloud compute instance-groups managed create fancy-fe-mig \
  --zone=$ZONE \
  --base-instance-name fancy-fe \
  --size 2 \
  --template fancy-fe
```

Copied!

content_copy

gcloud

compute

instance-

groups

managed

create

fancy-be-

mig \

--

zone=\$

ZONE \

--

base-

instan

ce-

name

fancy-

be \

--size

2 \

2. For your application, the frontend microservice runs on port 8080, and the backend microservice runs on port 8081 for orders and port 8082 for products:

```
gcloud compute instance-groups set-named-ports fancy-fe-mig \
  --zone=$ZONE \
  --named-ports frontend:8080
```

Copied!

```
content_copy
gcloud
compute
instance-
groups
set-named-
ports
fancy-be-
mig \
```

```
--
zone=$
ZONE \
--
named-
ports
orders
:8081
prod
```

Configure autohealing

To improve the availability of the application itself and to verify it is responding, configure an autohealing policy for the managed instance groups.

Since these are

an autohealing policy relies on an application-based health check to verify that an app is responding as expected. Checking that an app responds is more precise than simply verifying that an instance is in a RUNNING state, which is the default behavior.

named ports to identify these.

Note: Separate health checks for load balancing and for autohealing will be used. Health checks for load balancing can and should be more aggressive because these health checks determine whether an instance receives user traffic. You want to catch non-responsive instances quickly so you can redirect traffic if necessary.

Named ports are key-value pair metadata representing the service name and the port that it's running on. Named ports can be assigned to an instance group, which indicates that the service is available on all instances in the group.

In contrast, health checking for autohealing causes Compute Engine to proactively replace all bad instances, so this health check should be more conservative than a load balancing health check will be configured later.

1. Create a health check that repairs the instance if it returns "unhealthy" 3 consecutive times for the frontend and backend:

```
gcloud compute health-checks create http fancy-fe-hc \
  --port 8080 \
  --check-interval 30s \
  --healthy-threshold 1 \
  --timeout 10s \
```


CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
--unhealthy-threshold 3
```

Copied!

```
content_copy
gc cloud
compute
health-
checks
create
http
fancy-be-
hc \
```

```
--port
```

```
8081 \
```

```
--
```

```
reques
```

```
t-
```

```
path=/
```

```
api/
```

```
orders
```

```
\
```

```
--
```

```
check-
```

```
interv
```

```
al 30s
```

```
\
```

```
--
```

```
health
```

```
y-
```

```
thresh
```

```
old
```

```
1 \
```

```
--
```

```
timeou
```

```
t
```

```
10s \
```

```
--
```

```
unheal
```

```
thy-
```

```
thresh
```

```
old 3
```

Copied!

```
content_copy
```

2. Create a firewall rule to allow the health check probes to connect to the microservices on ports 8080-8081:

Create managed instance groups

Check my progress

```
--source-ranges 130.211.0.0/22,35.191.0.0/16 \
```

```
--network default
```

Copied!

```
content_copy
```

3. Appl

y

the

heal

th

chec

ks to

their

resp

Task 6. Create load balancers

To complement your managed instance groups, use HTTP(S) Load Balancers to serve traffic to the frontend and backend microservices, and use mappings to send traffic to the proper backend services based on pathing rules. This exposes a single load balanced IP for all services.

You can learn more about the Load Balancing options on Google Cloud: [Overview of Load Balancing](#).

Create HTTP(S) load balancer

Google Cloud offers many different types of load balancers. For this lab you use an HTTP(S) Load Balancer for your traffic. An HTTP load balancer is structured as follows:

1. A forwarding rule directs incoming requests to a target HTTP proxy.
2. The target HTTP proxy checks each request against a URL map to determine the appropriate backend service for the request.
3. The backend service directs each request to an appropriate backend based on serving capacity, zone, and instance health of its attached backends. The health of each backend instance is verified using an HTTP health check. If the backend service is configured to use an HTTPS or HTTP/2 health check, the request will be encrypted on its way to the backend instance.
4. Sessions between the load balancer and the instance can use the HTTP, HTTPS, or HTTP/2 protocol. If you use HTTPS or HTTP/2, each instance in the backend services must have an SSL certificate.

Note: For demonstration purposes in order to avoid SSL certificate complexity, use HTTP instead of HTTPS. For production, it is recommended to use HTTPS for encryption wherever possible.

5. Create health checks that will be used to determine which instances are capable of serving traffic for each service:

```
gcloud compute http-health-checks create fancy-fe-frontend-hc \
  --request-path / \
  --port 8080
```

Copied!

content_copy

```
gcloud
compute
http-
health-
checks
create
fancy-be-
orders-
hc \
--
request
```


CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
gcloud compute http-health-checks create fancy-be-products-hc \
  --request-path /api/products \
  --port 8082
```

Copied!

content_copy

Note: These health checks are for the load balancer, and only handle directing traffic from the load balancer; they do not cause the managed instance groups to recreate instances.

2. Create backend services that are the target for load-balanced traffic. The backend services will use the health checks and named ports you created:

```
gcloud compute backend-services create fancy-fe-frontend \
  --http-health-checks fancy-fe-frontend-hc \
  --port-name frontend \
  --global
```

Copied!

content_copy

```
gcloud
compute
backend-
services
create
fancy-be-
orders \
  --http-
health-
checks
fancy-
be-
orders-
hc \
  --port-
name
orders \
  --global
```

Copied!

content_copy

```
gcloud
compute
backend-
services
create
fancy-be-
products \
  --http-
health-
checks
fancy-
be-
products
-hc \
  --port-
name
products
\
  --global
```

Copied!

content_copy

3. Add the

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
--default-service fancy-fe-frontend
```

Copied!

content_copy

5. Create a path matcher to allow the /api/orders and /api/products paths to route to their respective services:

```
gcloud compute url-maps add-path-matcher fancy-map \  
  --default-service fancy-fe-frontend \  
  --path-matcher-name orders \  
  --path-rules "/api/orders=fancy-be-orders,/api/products=fancy-be-  
products"
```

Copied!

content_copy

6. Create the proxy which ties to the URL map:

```
gcloud  
compute  
target-  
http-  
Create HTTP(S) load balancers
```

Check my progress

```
fancy-  
proxy \  
  --url-  
map
```

Update the configuration

Now that you have a new static IP address, update the code on the frontend to point to this new address instead of the ephemeral address used earlier that pointed to the backend instance.

```
gcloud compute forwarding-rules create fancy-http-rule \  
  --global \  
  --target-http-proxy fancy-proxy \  
  --ports 80
```

Copied!

content_copy with-to-microservices/react-app/

Click **Check my progress** to verify the objective.

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

content_copy

2. Find the IP address for the Load Balancer:

```
gcloud compute forwarding-rules list --global
```

Copied!

content_copy

Example

```
NAME: fancy-http-rule
REGION:
IP_ADDRESS: 34.111.203.235
IP_PROTOCOL: TCP
TARGET: fancy-proxy
```

3. Return to the Cloud Shell Editor and edit the .env file again to point to Public IP of Load Balancer. [LB_IP] represents the External IP address of the backend instance determined above.

```
REACT_APP_ORDERS_URL=http://[LB_IP]/api/orders
REACT_APP_PRODUCTS_URL=http://[LB_IP]/api/products
```

Note: The ports are removed in the new address because the load balancer is configured to handle this forwarding for you.

4. **Save** the file.

5. Rebuild react-app, which will update the frontend code:

```
cd ~/monolith-to-microservices/react-app
npm install && npm run-script build
```

Copied!

content_copy

6. Copy the application code into your bucket:

```
cd ~
rm -rf
monolith-to-microservices/*/node_modules
gsutil -m cp -r monolith-to-
microservices gs://fancy-store-
$DEVSHELL_PROJECT_ID/
```

Copied!

content_copy

Update the frontend instances

Now that there is new code and configuration, you want the frontend instances within the managed instance group to pull the new code.

- ↗ Since your instances pull the code at startup, you can issue a rolling restart command:

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
gcloud compute instance-groups managed rolling-action replace fancy-fe-mig \
```

```
--zone=$ZONE \
--max-unavailable 100%
```

Copied!

content_copy

Note: In this

example of a

rolling

replace, you

specifically

state that all

machines can

Update the frontend instances

Check my progress

the command would keep an instance alive while restarting others to ensure availability. For testing purposes, you specify to replace all immediately for speed.

Click **Check my progress** to verify the objective.

Test the website

1. Wait 3 minutes after issuing the rolling-action replace command in order to give the instances time to be processed, and then check the status of the managed instance group. Run the following to confirm the service is listed as **HEALTHY**:

```
watch -n 2 gcloud compute backend-services get-health fancy-fe-frontend --global
```

Copied!

content_copy

2. Wait until

```
backend: https://www.googleapis.com/compute/v1/projects/my-gce-codelab/zones/us-central1-a/instanceGroups/fancy-fe-mig
status:
healthStatus:
- healthState: HEALTHY
  instance: https://www.googleapis.com/compute/v1/projects/my-gce-codelab/zones/us-central1-a/instances/fancy-fe-x151
  ipAddress: 10.128.0.7
  port: 8080
- healthState: HEALTHY
  instance: https://www.googleapis.com/compute/v1/projects/my-gce-codelab/zones/us-central1-a/instances/fancy-fe-cgrt
  ipAddress: 10.128.0.11
  port: 8080
kind: compute#backendServiceGroupHealth
```

Note: If one instance encounters an issue and is UNHEALTHY it should automatically be repaired. Wait for this to happen.

If neither instance enters a HEALTHY state after waiting a little while, something is wrong with the setup of the frontend instances that accessing them on port 8080 doesn't work. Test this by browsing to the instances directly on port 8080.

3. Once both items appear as HEALTHY on the list, exit the watch command by pressing CTRL+C.

Note: The application will be accessible via `http://[LB_IP]` where [LB_IP] is the IP_ADDRESS specified for the Load Balancer, which can be found with the following command:

```
gcloud compute forwarding-rules list --global
```

You'll be checking the application later in the lab.

Task 7. Scaling Compute Engine

So far, you have created two managed instance groups with two instances each. This configuration is fully functional, but a static configuration regardless of load. Next, you create an autoscaling policy based on utilization to automatically scale each managed instance group.

Automatically resize by utilization

- ✎ To create the autoscaling policy, execute the following:

```
gcloud compute instance-groups managed set-autoscaling \
  fancy-fe-mig \
  --zone=$ZONE \
  --max-num-replicas 2 \
  --target-load-balancing-utilization 0.60
```

Copied!

```
content_copy
gcloud
compute
instance-
groups
managed
set-
autoscalin
g \
  fancy-
be-mig \
  --
zone=$ZO
NE \
```


CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

Copied!

content_copy

These

command

s create

an

autoscaler

on the

managed

instance

group that

enables

Enable content delivery network

Another feature that can help with scaling is to enable a Content Delivery Network service, to provide caching for the frontend, and removes instances when the load balancer is below 60% utilization.

1. Execute the following command on the frontend service:

```
gcloud compute backend-services update fancy-fe-frontend \
  --enable-cdn --global
```

Copied!

content_copy

When a

user

requests

content

from the

HTTP(S)

load

balancer,

the

request

arrives at

a Google Front End (GFE) which first looks in the Cloud CDN cache for a response to the user's request. If the GFE finds a cached response, the GFE

Scaling Compute Engine

Check my progress

If the GFE can't find a cached response for the request, the GFE makes a request directly to the backend. If the response to this request is cacheable, the GFE stores the response in the Cloud CDN cache so that the cache can be used for subsequent requests.

Click **Check my progress** to verify the objective.

Task 8. Update the website

Updating instance template

Existing instance templates are not editable; however, since your instances are stateless and all configuration is done through the startup script, you only need to change the instance template if you want to change the template settings. Now you're going to make a simple change to use a larger machine type and push that out.

Complete the following steps to:

- ↗ Update the frontend instance, which acts as the basis for the instance template. During the update, put a file on the updated version of the instance template's image, then update the instance template, roll out the new template, and then confirm the file exists on the managed instance group instances.
- ↗ Modify the machine type of your instance template, by switching from the e2-standard-2 machine type to e2-small.

1. Run the following command to modify the machine type of the frontend instance:

```
gcloud compute instances set-machine-type frontend \
  --zone=$ZONE \
  --machine-type e2-small
```

Copied!

content_copy

2. Create the new Instance Template:

```
gcloud compute instance-templates create fancy-fe-new \
  --region=$REGION \
  --source-image=debian-9
```


CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

Copied!
content_copy
This will
take a few
moments.

Once you
have at
least 1
instance in
the
following
condition:

STATUS:
RUNNING

ACTION
set to
None

INSTANCE_
TEMPLATE:
the new
template

name
machineType: <https://www.googleapis.com/compute/v1/projects/project-name/zones/us-central1-f/machineTypes/e2-small>
new)

5. Copy Make changes to the website

Scenario: Your marketing team has asked you to change the homepage for your site. They think it should be more informative of who your company is and what you actually sell.

Task: Add some text to the homepage to make the marketing team happy! It looks like one of the developers has already created the changes with the file `index.js.new`. You can just copy this file to `index.js` and the changes should be reflected. Follow the instructions below to make the appropriate changes.

1. Run the following commands to copy the updated file to the correct file name:

```
cd ~/monolith-to-microservices/react-app/src/pages/Home
```


CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
mv index.js.new index.js
```

Copied!

content_copy

2. Print

the

file

cont

ents

to

verif

y

the

chan

ges:

```
cat
```

```
~/monolith
```

```
-to-
```

```
microservi
```

```
ces/react-
```

```
app/src/pa
```

```
ges/Home/i
```

```
ndex.js
```

Copied!

content_copy

The

resulting

code

should

look like

this:

```
/*
```

```
Copyright
```

```
2019
```

```
Google LLC
```

```
Licensed under the Apache License, Version 2.0 (the "License");
```

```
you may not use this file except in compliance with the License.
```

```
You may obtain a copy of the License at
```

```
https://www.apache.org/licenses/LICENSE-2.0
```

```
Unless required by applicable law or agreed to
```

```
in writing, software
```

```
distributed under the License is distributed on an "AS IS" BASIS,
```

```
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
```

```
See the License for the specific language governing permissions and
```

```
limitations under the License.
```

```
*/
```

```
import React from "react";
```

```
import { Box, Paper, Typography } from "@mui/material";
```

```
export default function Home() {
```

```
  return (
```

```
    <Box sx={{ flexGrow: 1 }}>
```

```
      <Paper
```

```
        elevation={3}
```

```
        sx={{
```

```
          width: "800px",
```

```
          margin: "0 auto",
```

```
          padding: (theme)
```


CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
npm install && npm run-script build
Copied!
```

```
content_copy
```

4.

Then

re-push

this

code to

the

bucket:

```
cd ~
```

```
rm -rf
```

```
monolith-
```

Push changes with rolling

replacements

1. Now force all instances to be replaced to pull the update:

```
cp -r
```

```
monolith gcloud compute instance-groups managed rolling-action replace fancy-fe-
```

```
fig \
```

```
microservices $ZONE \
```

```
cs --max-unavailable=100%
```

```
gs://fancy
```

```
-store-
```

```
content_copy
```

```
$DEVSHLL
```

```
PROJECT_ID
```

example of a

rolling!

replace, you

content_copy

specifically

state that all

machines can

be

replaced immediately through the `--max-unavailable` parameter. Without this parameter,

Update the website

Check my progress

2. Wait 3 minutes after issuing the rolling action replace command in order to give the instances time to be processed and then check

Click **Check my progress** storage in the objective

confirm the service is listed

as **HEALTHY**:

```
watch -n 2 gcloud compute backend-services get-health fancy-fe-frontend
```

```
--global
```

Copied!

```
content_copy
```

3. Wait a few moments for both services to appear and become HEALTHY. Example output:

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
backend: https://www.googleapis.com/compute/v1/projects/my-gce-
code-lab/zones/us-central1-a/instanceGroups/fancy-fe-mig
status:
healthStatus:
- healthState: HEALTHY
  instance: https://www.googleapis.com/compute/v1/projects/my-gce-
code-lab/zones/us-central1-a/instances/fancy-fe-x151
  ipAddress: 10.128.0.7
  port: 8080
- healthState: HEALTHY
  instance: https://www.googleapis.com/compute/v1/projects/my-gce-
code-lab/zones/us-central1-a/instances/fancy-fe-cgrt
  ipAddress: 10.128.0.11
  port: 8080
kind: compute#backendServiceGroupHealth
```

4. Once items appear in the list with HEALTHY status, exit the watch command by pressing CTRL+C.
5. Browse to the website via `http://[LB_IP]` where [LB_IP] is the IP_ADDRESS specified for the Load Balancer, which can be found with the following command:

```
gcloud compute forwarding-rules list --global
```

Copied!

content_copy

The new

website

changes

should

now be
visible.
**Simulate
failure**

In order to confirm the health check works, log in to an instance and stop the services.

1. To find an instance name, execute the following:

```
gcloud compute instance-groups list-instances fancy-fe-mig --zone=$ZONE
```

Copied!

content_copy

2. Copy an instance name, then run the following to secure shell into the instance, where INSTANCE_NAME is one of the instances from the list:

```
gcloud compute ssh [INSTANCE_NAME] --zone=$ZONE
```

Copied!

content_copy

3. Type
in
"y"
to
confi
rm,
and
pres

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

```
sudo supervisorctl stop nodeapp; sudo killall node
```

Copied!

content_copy

5. Exit
the
insta
nce:

```
exit
```

Copied!

content_copy

6. Moni
tor
the
repa
ir
oper
atio
ns:

NAME	TYPE
TARGET	HTTP_STATUS STATUS
repair-1568314034627-5925f90ee238d-fe645bf0-7becce15	
compute.instances.repair.recreateInstanceus-central1- a/instances/fancy- fe-1vqq	200 DONE 2019-09-12T11:47:14.627- 07:00

The managed instance group recreated the instance to repair it.

--

```
filter='op  
erationTyp
```

7. You can also go to **Navigation menu > Compute Engine > VM instances** to monitor through the Console.

```
e~compute  
instances.  
repair.*'
```

Copied!

content_copy

This will
take a few
minutes to
complete.

Congratulations!

the

following

You have successfully deployed, scaled, and updated your website on Compute Engine. You are now experienced with Compute Engine, Managed Instance Groups, Load Balancers, and Health Checks!

Source :

<https://www.cloudskillsboost.google/focuses/11952?parent=catalog>

Creating an application in the App Engine flexible environment

Cloud Tools for Eclipse provides a wizard inside Eclipse to create new applications for the App Engine flexible environment.

Before you begin

1. In the Google Cloud console, on the project selector page, select or create a Google Cloud project.

Note: If you don't plan to keep the resources that you create in this procedure, create a project instead of selecting an existing project. After you finish these steps, you can delete the project, removing all resources associated with the project.

Go to project selector

2. Make sure that billing is enabled for your Google Cloud project.
3. Enable the Cloud Build API.

Enable the API

4. Install the Google Cloud CLI.
5. To initialize the gcloud CLI, run the following command:

```
gcloud init
```

6. Create an App Engine application for your Google Cloud project in the Google Cloud console.

Open app creation

7. Select a region where you want your app's computing resources located.

Note: After you create your App Engine app, you cannot change the region. To reduce latency, choose the region closest to your app's intended users. For more information on the available regions, see App Engine Locations.

8. Ensure you have the latest version of Cloud Tools for Eclipse.

Creating a flex application

To create an application for the App Engine flexible environment:

1. Click the **Google Cloud Platform** toolbar button .
2. Select **Create New Project > Google App Engine Flexible Java Project....**

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

The New App Engine Flexible Project wizard displays

New App Engine Flexible Project

App Engine Flexible Project
Create a new Eclipse project for App Engine flexible environment development.

Project name: flexproject

☒ Use default location

Location: /home/elharo/workspaces/qa6/flexproject Browse...

Java package: com.example

App Engine service: default

☐ Create as Maven project

Maven project coordinates

Group ID:

Artifact ID:

Version: 0.1.0-SNAPSHOT

? < Back Next > Cancel Finish

3. Enter a **Project name** for your application.
4. If you want, enter values for the optional fields:
 - a. Enter a **Java package** name.
 - b. Select the **Create as Maven project** checkbox to generate a pom.xml and configure the Eclipse project to use the Maven builder.
 - c. Enter the **Group ID** and **Artifact ID** for your Maven project.
5. Click **Next**.
6. Select any libraries you need in the project.
7. Click **Finish**.

The new application appears in the Eclipse **Project Explorer**.

Source: <https://cloud.google.com/eclipse/docs/create-flex-app>

Exploring PAAS with APP ENGINE

It is a fully managed PaaS (platform as a service) cloud computing platform that uses in-built services to run your apps. You can start creating almost immediately after receiving the software development kit (SDK). You may immediately access the Google app developer's manual once you've chosen the language you wish to use to build your app.

After creating a Cloud account, you may Start Building your App

- ↗ Using the Go template/HTML package
- ↗ Python-based webapp2 with Jinja2
- ↗ PHP and Cloud SQL
- ↗ using Java's Maven

The app engine runs the programmers on various servers while handling the code. The app engine powers programs like Snapchat, Rovio, and Khan Academy.

Features of App Engine

Runtimes and Languages

To create an application for an app engine, you can use Go, Java, PHP, or Python. You can develop and test an app locally using the SDK's deployment toolkit. Each language's SDK and run time are unique. Your program is run in a:

- ↗ Java Run Time Environment version 7
- ↗ Python Run Time environment version 2.7
- ↗ PHP runtime's PHP 5.4 environment
- ↗ Go runtime 1.2 environment

Generally Usable Features

These are protected by the service-level agreement and depreciation policy of the app engine. The implementation of such a feature is often stable, and any changes made to it are backward-compatible. These include communications, process management, computing, data storage, retrieval, and search, as well as app configuration and management. Features like the HRD migration tool, Google Cloud SQL, logs, datastore, dedicated Memcached, blob store, Memcached, and search are included in the categories of data storage, retrieval, and search.

Features in Preview

In a later iteration of the app engine, these functions will undoubtedly be made broadly accessible. However, because they are in the preview, their implementation may change in ways that are backward-incompatible. Sockets, MapReduce, and the Google Cloud Storage Client Library are a few of them.

Experimental Features

These might or might not be made broadly accessible in the next app engine updates. They might be changed in ways that are irreconcilable with the past. The “trusted tester” features, however, are only accessible to a limited user base and require registration in order to utilize them. The experimental features include Prospective Search, Page Speed, OpenID, Restore/Backup/Datastore Admin, Task Queue Tagging, MapReduce, and Task Queue REST API. App metrics analytics, datastore admin/backup/restore, task queue tagging, MapReduce, task queue REST API, OAuth, prospective search, OpenID, and Page Speed are some of the experimental features.

Third-Party Services

As Google provides documentation and helper libraries to expand the capabilities of the app engine platform, your app can perform tasks that are not built into the core product you are familiar with as app engine. To do this, Google collaborates with other organizations. Along with the helper libraries, the partners frequently provide exclusive deals to app engine users.

Event Driven Programs with Cloud Functions

Write event-driven functions

In Cloud Functions, you use event-driven functions when you want a function to be invoked automatically in response to an event that occurs in your cloud environment.

There are two ways to implement event-driven functions. Which way you use depends on your chosen language runtime and whether you are using Cloud Functions (1st gen) or Cloud Functions (2nd gen):

- ↗ In Cloud Functions (2nd gen), for all runtimes, use CloudEvent functions.
- ↗ In Cloud Functions (1st gen):
- ↗ For Node.js, Python, Go, and Java runtimes, use Background functions.
- ↗ For .NET, Ruby, and PHP runtimes, use CloudEvent functions.

CloudEvent functions

CloudEvent functions are based on CloudEvents, an industry-standard specification for describing event data in a common way. You can learn more about the CloudEvents specification at the CloudEvents GitHub repository. The CloudEvents project also provides a set of CloudEvents SDKs to help work with CloudEvents objects in your code.

The following example shows a basic CloudEvent function source file for each runtime. See Source directory structure for information about where to locate your source code.

Node.js Python Go Java C# Ruby PHP

```
const functions = require('@google-cloud/functions-framework');

// Register a CloudEvent function with the Functions Framework
functions.cloudEvent('myCloudEventFunction', cloudEvent => {
  // Your code here
  // Access the CloudEvent data payload via cloudEvent.data
});
```

In Node.js, you register a CloudEvent handler function with the Functions Framework for Node.js. Your handler function must accept a CloudEvent object as an argument.

The function entry point is the name with which the handler is registered with the Functions Framework. In this example, the entry point is `myCloudEventFunction`.

For CloudEvent functions, event data is passed to your function in the CloudEvents format, with a CloudEvent data payload corresponding to the event type that triggers your function. See Cloud Functions triggers for information about supported triggers, event types, and associated event data formats.

The Google Events repository contains resources for working with CloudEvents issued by Google.

Background functions

Event-driven functions in the Cloud Functions (1st gen) Node.js, Python, Go, and Java runtimes expect different arguments from CloudEvent functions. This older style of event-driven function is called a *background function*.

The following example shows a basic background function source file for each runtime. See [Source directory structure](#) for information about where to locate your source code.

Node.js Python Go Java

```
// Define and export an event handler
exports.myBackgroundFunction = (eventData, context, callback) => {
  // Your code here
  // The eventData argument represents the event data payload

  // Optionally signal function completion:
  callback();
};
```

In Node.js, you define and export a function that handles event data. Cloud Functions passes your handler function the following arguments:

- ↗ eventData: An object representing the event data payload. Its format depends on the event type.
- ↗ context: An object containing metadata about the event.
- ↗ callback: An optional function you can call to signal completion. The first argument to this callback is interpreted as signaling an error. Pass no arguments or a null first argument to signal success.

The function entry point is the name of the exported event handler. In this example, the entry point is `myBackgroundFunction`.

For background functions, the event data payload is passed directly to your function in a format corresponding to the event type that triggers your function. See [Triggers supported in Cloud Functions \(1st gen\)](#) for information about supported triggers, event types, and associated event data formats.

Function termination

Cloud Functions considers event-driven function execution complete when the function returns. If the function creates background tasks (such as with threads, futures, JavaScript Promise objects, callbacks, or system processes), you must terminate or otherwise resolve these tasks before returning from your function. Any tasks not terminated before the function returns might not be completed, and might cause undefined behavior.

Automatic retries

Event-driven functions can be configured to automatically retry failed invocations. See [Retrying event-driven functions](#) for more information.

SOURCE

<https://cloud.google.com/functions/docs/writing/write-event-driven-functions>

GKE overview

This page describes Google Kubernetes Engine (GKE), a managed [Kubernetes](#) service that you can use to deploy and operate containerized applications at scale using Google's infrastructure. This page is intended for platform administrators who are looking for a scalable, automated, managed Kubernetes solution. You should already be familiar with [Kubernetes concepts](#).

GKE is a Google-managed implementation of the [Kubernetes](#) open source container orchestration platform. Kubernetes was developed by Google, drawing on years of experience operating production workloads at scale on [Borg](#), our in-house cluster management system.

When to use GKE

GKE is ideal if you need a platform that lets you configure the infrastructure that runs your containerized apps, such as networking, scaling, hardware, and security. GKE provides the operational power of Kubernetes while managing many of the underlying components, such as the control plane and nodes, for you.

Benefits of GKE

The following table describes some of the benefits of using GKE as your managed Kubernetes platform:

GKE benefits

Platform management	Fully-managed nodes in GKE Autopilot mode with built-in hardening and best practice configurations automatically applied. Managed upgrade experience with release channels to improve security, reliability, and compliance.
---------------------	---

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

	Flexible <u>maintenance windows and exclusions</u> that let you configure upgrade type and scope to meet business needs and architecture constraints. In GKE Standard mode, flexible <u>node upgrade strategies</u> to optimize availability and manage disruptions. Automatic scaling of nodes based on the number of Pods in the cluster with Autopilot mode or with node auto-provisioning in Standard mode. <u>Node auto-repair</u> to maintain node health and availability. Built-in <u>logging and monitoring</u>. Google Cloud integrated CI/CD options with <u>Cloud Build</u> and <u>Cloud Deploy</u>.
Improved security posture	Hardened node operating system for apps: <u>Container-Optimized OS</u>. <u>Built-in security measures</u>. <u>Automatic upgrades</u> to new GKE versions. Integrated security posture monitoring tooling with the <u>security posture dashboard</u>.
Cost optimization	Google Cloud logging and monitoring integrations with <u>Google Cloud's operations suite</u>. In Autopilot mode, pay only for the compute resources your running Pods request. In GKE Standard mode, you pay for all resources on nodes, regardless of Pod requests. Save costs by running fault-tolerant workloads, such as batch jobs, on <u>Spot Pods</u>.
Reliability and availability	Minimized operational overhead in Autopilot mode because Google manages both the nodes and the control plane. >99% <u>monthly uptime SLO</u>. Pod-level SLA in Autopilot clusters because Google manages the nodes. Highly-available control plane and worker nodes in <u>Autopilot mode</u> and in <u>regional Standard clusters</u>. <u>Proactive monitoring and recommendations</u> to mitigate potential workload disruptions caused by upcoming deprecations. Multi-cluster Service capabilities.

Use cases for GKE

GKE and Kubernetes are used in a variety of industries, including robotics, healthcare, retail, education, gaming, and financial services. The range of applications include AI and ML operations, data processing at scale, operating scalable online games platforms, and running reliable applications under heavy load.

For case studies by industry and application, refer to [Google Cloud customers](#).

How GKE works

A GKE environment consists of *nodes*, which are Compute Engine virtual machines (VMs), that are grouped together to form a *cluster*. You package your apps (also called *workloads*) into containers. You deploy sets of containers as *Pods* to your nodes. You use the Kubernetes API to interact with your workloads, including administering, scaling, and monitoring.

Note: If you run your workloads directly in VMs instead of containers, use Migrate for GKE to containerize the workloads without having to rewrite apps or modify source code.

Kubernetes clusters have a set of management nodes called the *control plane*, which run system components such as the Kubernetes API server. In GKE, Google manages the control plane and system components for you. In Autopilot mode, which is the recommended way to run GKE, Google also manages your worker nodes. Google automatically upgrades component versions for improved stability and security, ensuring high availability, and ensuring integrity of data stored in the cluster's persistent storage.

For more information, refer to [GKE cluster architecture](#).

Kubernetes versions and features

GKE automatically upgrades your control plane to new Kubernetes versions that add new features and improvements in the open source project. The Kubernetes version selected for auto-upgrades depends on the stable version in the GKE release channel you select when you create the cluster. You can also manually upgrade your control plane to a different Kubernetes version than the version GKE selects for an upgrade.

For detailed information on versions and upgrades, refer to the [release notes](#) and [GKE versioning and upgrades](#). If you use GKE Standard mode and don't enroll in a release channel, you won't get automatic upgrades.

GKE includes most *beta* and *stable* Kubernetes features. If you want to try less stable Kubernetes features in the *alpha* stage, use [alpha Standard clusters](#).

Modes of operation

GKE has the Autopilot and Standard *modes of operation*, which offer you different levels of flexibility, responsibility, and control. We recommend the fully-managed Autopilot mode, in which Google Cloud manages your nodes for you and provides a workload-focused, cost-optimized, production-ready experience.

If you want more information before you choose a mode, refer to [Choose a GKE mode of operation](#).

Deploying a containerized web application

bookmark_border

[AUTOPILOT](#) [STANDARD](#)

This tutorial shows you how to package a web application in a Docker container image, and run that container image on a Google Kubernetes Engine (GKE) cluster. Then, you deploy the web application as a load-balanced set of replicas that can scale to the needs of your users.

Objectives

- ↗ Package a sample web application into a Docker image.
- ↗ Upload the Docker image to Artifact Registry.
- ↗ Create a GKE cluster.
- ↗ Deploy the sample app to the cluster.
- ↗ Manage autoscaling for the deployment.
- ↗ Expose the sample app to the internet.
- ↗ Deploy a new version of the sample app.

Costs

In this document, you use the following billable components of Google Cloud:

- ↗ [GKE](#)
- ↗ [Artifact Registry](#)

To generate a cost estimate based on your projected usage, use the [pricing calculator](#).

When you finish the tasks that are described in this document, you can avoid continued billing by deleting the resources that you created. For more information, see [Clean up](#).

Before you begin

1. In the Google Cloud console, on the project selector page, select or [create a Google Cloud project](#).

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

Note: If you don't plan to keep the resources that you create in this procedure, create a project instead of selecting an existing project. After you finish these steps, you can delete the project, removing all resources associated with the project.

Go to project selector

2. Make sure that billing is enabled for your Google Cloud project.
3. Enable the Compute Engine, Artifact Registry, and Google Kubernetes Engine APIs.

Enable the APIs

Activate Cloud Shell

Cloud Shell comes preinstalled with the `gcloud`, `docker`, and `kubectl` command-

1. Go to the Google Cloud console.
 button at the top of the Google Cloud console
2. Click the **Activate Cloud Shell** console window.

A Cloud Shell session opens inside a new frame at the bottom of the Google Cloud console and displays a command-line prompt.



Create a repository

In this tutorial, you store an image in Artifact Registry and deploy it from the registry. For this quickstart, you'll create a repository named `hello-repo`.

1. Set the `PROJECT_ID` environment variable to your Google Cloud project ID (`PROJECT_ID`). You'll use this environment variable when you build the container image and push it to your repository.

```
export PROJECT_ID=PROJECT_ID
```

2. Confirm that the `PROJECT_ID` environment variable has the correct value:

```
echo $PROJECT_ID
```

3. Set your project ID for the Google Cloud CLI:


```
gcloud config set project $PROJECT_ID
```

Output:

Updated property [core/project].

Note: Your gcloud CLI might already be set to your `$PROJECT_ID`—you will still see the **Updated property** output.

4. Create the `hello-repo` repository with the following command:

```
gcloud artifacts repositories create hello-repo \
  --repository-format=docker \
  --location=REGION \
  --description="Docker repository"
```

Replace `REGION` with the a region for the repository, such as `us-west1`. To see a list of available locations, run the command:

```
gcloud artifacts locations list
```

Building the container image

In this tutorial, you deploy a sample web application called `hello-app`, a web server written in Go that responds to all requests with the message `Hello, World!` on port 8080.

GKE accepts Docker images as the application deployment format. Before deploying `hello-app` to GKE, you must package the `hello-app` source code as a Docker image.

To build a Docker image, you need source code and a Dockerfile. A Dockerfile contains instructions on how the image is built.

1. Download the `hello-app` source code and Dockerfile by running the following commands:

```
git clone https://github.com/GoogleCloudPlatform/kubernetes-engine-samples
cd kubernetes-engine-samples/hello-app
```

2. Build and tag the Docker image for `hello-app`:

```
docker build -t REGION-docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v1 .
```


This command instructs Docker to build the image using the Dockerfile in the current directory, save it to your local environment, and tag it with a name, such as `us-west1-docker.pkg.dev/my-project/hello-repo/hello-app:v1`. The image is pushed to Artifact Registry in the next section.

- ↗ The `PROJECT_ID` variable associates the container image with the `hello-repo` repository in your Google Cloud project.
- ↗ The `us-west1-docker.pkg.dev` prefix refers to Artifact Registry, regional host for your repository.

Note: If prompted, authorize Cloud Shell to make Google Cloud API calls.

3. Run the `docker images` command to verify that the build was successful:

```
docker images
```

Output:

REPOSITORY	TAG	IMAGE
ID	CREATED	SIZE
us-west1-docker.pkg.dev/my-project/hello-repo/hello-app	v1	
25cfadb1bf28	10 seconds ago	54 MB

Note: You might also see additional images that represent intermediate image layers.

4. Add IAM policy bindings to your service account:

```
gcloud artifacts repositories add-iam-policy-binding hello-repo \
  --location=REGION \
  --member=serviceAccount:PROJECT_NUMBER-
compute@developer.gserviceaccount.com \
  --role="roles/artifactregistry.reader"
```

Replace `PROJECT_NUMBER` with the project number of your project.

Running your container locally (optional)

1. Test your container image using your local Docker engine:

```
docker run --rm -p 8080:8080 REGION-docker.pkg.dev/${PROJECT_ID}/hello-
repo/hello-app:v1
```

Pushing the Docker image to Artifact Registry

2. Click the **Web Preview** button and then select the 8080 port number. GKE opens the preview URL on its proxy service in a new browser window.

You must upload the container image to a registry so that your GKE cluster can download and run the container image. In this tutorial, you will store your container in Artifact Registry.

1. Configure the Docker command-line tool to authenticate to Artifact Registry:

```
gcloud auth configure-docker REGION-docker.pkg.dev
```

Note: You can ignore any warnings about having a *long list of credential helpers*.

2. Push the Docker image that you just built to the repository:

```
docker push REGION-docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v1
```

Creating a GKE cluster

Now that the Docker image is stored in Artifact Registry, create a GKE cluster to run hello-app. A GKE cluster consists of a pool of Compute Engine VM

instances running Kubernetes, the open source cluster orchestration system that powers GKE.

Cloud Shell Console

3. Set your Compute Engine region:

```
gcloud config set compute/region REGION
```

For Standard zonal clusters, set a Compute Engine zone nearest to the Artifact Registry repository.

2. Create a cluster named hello-cluster:

```
gcloud container clusters create-auto hello-cluster
```

It takes a few minutes for your GKE cluster to be created and health-checked. To run this tutorial on a GKE Standard cluster, use the gcloud container clusters create command instead.

Deploying the sample app to GKE

You are now ready to deploy the Docker image you built to your GKE cluster.

Kubernetes represents applications as Pods, which are scalable units holding one or more containers. The Pod is the smallest deployable unit in Kubernetes. Usually, you deploy Pods as a set of replicas that can be scaled and distributed together across your cluster. One way to deploy a set of replicas is through a Kubernetes Deployment.

In this section, you create a Kubernetes Deployment to run hello-app on your cluster. This Deployment has replicas (Pods). One Deployment Pod

container: the hello-app Docker image. You also create a HorizontalPodAutoscaler resource that scales the number of Pods from 3 to a number between 1 and 5, based on CPU load.

Cloud Shell Console

1. Ensure that you are connected to your GKE cluster.

```
gcloud container clusters get-credentials hello-cluster --region  
REGION
```

2. Create a Kubernetes Deployment for your hello-app Docker image.

```
kubectl create deployment hello-app --image=REGION-  
docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v1
```

3. Set the baseline number of Deployment replicas to 3.

```
kubectl scale deployment hello-app --replicas=3
```

4. Create a HorizontalPodAutoscaler resource for your Deployment.

```
kubectl autoscale deployment hello-app --cpu-  
percent=80 --min=1 --max=5
```

5. To see the Pods created, run the following command:

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
hello-app-784d7569bc-hgmpx	1/1	Running	0	90s
hello-app-784d7569bc-jfkz5	1/1	Running	0	90s
hello-app-784d7569bc-mnr1l	1/1	Running	0	95s

Note: The Pods might take up to two minutes to start running. Until they start, the **status** column displays **Pending**.

Exposing the sample app to the internet

While Pods do have individually-assigned IP addresses, those IPs can only be reached from inside your cluster. Also, GKE Pods are designed to be ephemeral, starting or stopping based on scaling needs. And when a Pod crashes due to an error, GKE automatically redeploys that Pod, assigning a new Pod IP address each time.

What this means is that for any Deployment, the set of IP addresses corresponding to the active set of Pods is dynamic. We need a way to 1) group Pods together into one static hostname, and 2) expose a group of Pods outside the cluster, to the internet.

Kubernetes Services solve for both of these problems. Services group Pods into one static IP address, reachable from any Pod inside the cluster. GKE also assigns a DNS hostname to that static IP. For example, `hello-app.default.svc.cluster.local`.

The default Service type in GKE is called ClusterIP, where the Service gets an IP address reachable only from inside the cluster. To expose a Kubernetes Service outside the cluster, create a Service of type LoadBalancer. This type of Service spawns an External Load Balancer IP for a set of Pods, reachable through the internet.

In this section, you expose the `hello-app` Deployment to the internet using a Service of type `LoadBalancer`.

Note: External Application Load Balancers are (subject to billing) separate from GKE's costs.

Cloud Shell Console

1. Use the `kubectl expose` command to generate a Kubernetes Service for the `hello-app` deployment:

```
kubectl expose deployment hello-app --name=hello-app-service --  
type=LoadBalancer --port 80 --target-port 8080
```

Here, the `--port` flag specifies the port number configured on the Load Balancer, and the `--target-port` flag specifies the port number that the `hello-app` container is listening on.

2. Run the following command to get the Service details for `hello-app-service`:

Output:

NAME	CLUSTER-IP	EXTERNAL-IP	PORT(S)
hello-app-service	10.3.251.122	203.0.113.0	80:30877/TCP

10s

3. Copy the `EXTERNAL_IP` address to the clipboard (for instance: `203.0.113.0`).

Note: It might take a few minutes for the Load Balancer to be provisioned. Until the Load Balancer is provisioned, you might see a `<pending>` IP address.

Now that the `hello-app` Pods are exposed to the internet through a Kubernetes Service, you can open a new browser tab, and navigate to the Service IP address you copied to the clipboard. A Hello, World! message appears, along with a Hostname field.

The Hostname corresponds to one of the three `hello-app` Pods serving your HTTP request to your browser.

Deploying a new version of the sample app

In this section, you upgrade `hello-app` to a new version by building and deploying a new Docker image to your GKE cluster.

GKE's rolling update feature lets you update your Deployments without downtime. During a rolling update, your GKE cluster incrementally replaces the existing `hello-app` Pods with Pods containing the Docker image for the new version. During the update, your load balancer service routes traffic only into available Pods.

1. Return to Cloud Shell, where you have cloned the `hello app` source code and Dockerfile. Update the function `hello()` in the `main.go` file to report the new version `2.0.0`.
2. Build and tag a new `hello-app` Docker image.

```
docker build -t REGION-docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v2 .
```

3. Push the image to Artifact Registry.

```
docker push REGION-docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v2
```

Now you're ready to update your `hello-app` Kubernetes Deployment to use a new Docker image.

Note: Because you created a `HorizontalPodAutoscaler` earlier, the number of `hello-app` Pods running in this section might be fewer than the three Pods initially deployed. Your GKE cluster might have scaled down the number of Pods based on low CPU usage.

Cloud Shell Console

4. Apply a rolling update to the existing `hello-app` Deployment with an image update using the `kubectl set image` command:

```
kubectl set image deployment/hello-app hello-app=REGION-docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v2
```

2. ~~watch kubectl get pods~~ Watch the running Pods running the `v1` image stop, and new Pods running the `v2` image start.

Output:

NAME	READY	STATUS	RESTARTS	AGE
hello-app-89dc45f48-5bzqp	1/1	Running	0	2m42s
hello-app-89dc45f48-scm66	1/1	Running	0	2m40s

3. In a separate tab, navigate again to the `hello-app-service` External IP. You should now see the `Version` set to `2.0.0`.

Clean up

CLOUD COMPUTING-II

UNIT-II

Networking in the Cloud



Parul[®]
University

NAAC A++

To avoid incurring charges to your Google Cloud account for the resources used in this tutorial, either delete the project that contains the resources, or keep the project and delete the individual resources.

1. **Delete the Service:** This deallocates the Cloud Load Balancer created for your Service:

```
kubectl delete service hello-app-service
```

2. **Delete the cluster:** This deletes the resources that make up the cluster, such as the compute instances, disks, and network resources:

```
gcloud container clusters delete hello-cluster --region REGION
```

3. **Delete your container images:** This deletes the Docker images you pushed to Artifact Registry.

```
gcloud artifacts docker images delete \  
  REGION-docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v1 \  
  --delete-tags --quiet  
gcloud artifacts docker images delete \  
  REGION-docker.pkg.dev/${PROJECT_ID}/hello-repo/hello-app:v2 \  
  --delete-tags --quiet
```

Source:

<https://cloud.google.com/kubernetes-engine/docs/tutorials/hello-app>

Hands on : <https://www.youtube.com/watch?v=ZVcnsTCQASU>